

API SECURITY RISK ANALYSIS REPORT

public-apis / public-apis
Vulnerability Assessment | May 2026
Classification: CONFIDENTIAL

1. Executive Summary

This report presents the findings of a security risk analysis performed on the public-apis GitHub repository and its associated API validation infrastructure. Evidence was gathered from source code review (github_pull_request.sh), HTTP response analysis, and network timing data captured via API client tools.

A total of **five (5) security risks** were identified across the following categories: HTTP method misconfiguration, authentication bypass, information disclosure, credential exposure in CI/CD pipelines, and input validation weaknesses. The most critical finding is a **405 Method Not Allowed** error returned by the API endpoint, indicating that POST/PUT/DELETE operations are unsupported or misconfigured — a condition that can be exploited in method-override attacks.

Overall Risk Rating: **HIGH**

Total Risks	Critical	High	Medium/Low
5	1	2	2

2. Scope & Evidence Sources

- The assessment covered the following artefacts and evidence sources:
- GitHub Shell Script: scripts/github_pull_request.sh — CI/CD automation for PR validation
 - HTTP Response Body: 405 Method Not Allowed (HTML, unicorn/19.9.0)
 - HTTP Response Headers: Allow: OPTIONS, GET, HEAD; CORS: Access-Control-Allow-Origin: *
 - API Timing Telemetry: DNS 20ms, TLS 31ms, Wait 108ms, Total 172ms
 - HTTP Raw Message: Full request/response cycle captured via API client

3. Risk Register

All identified risks are documented below with severity ratings, technical impact, and remediation guidance.

Risk 1 — HTTP Method Misconfiguration (405 Method Not Allowed)

Category	Severity	Affected Component	CVSS Score	Evidence
HTTP Method Control	CRITICAL	API Gateway / Backend Server	7.5 (High)	Images 2, 4, 5

Description:

The API server returns HTTP 405 Method Not Allowed for certain request types. The Allow header explicitly lists only OPTIONS, GET, and HEAD, meaning write operations (POST, PUT, DELETE, PATCH) are entirely blocked at the server level. While this may be intentional for a read-only API, the gunicorn/19.9.0 server version is publicly exposed, and the error handling reveals internal server architecture.

Impact:

- Attackers can enumerate supported HTTP methods via the Allow header
- Server version disclosure (gunicorn/19.9.0) enables version-targeted exploits
- Method override attacks (X-HTTP-Method-Override header) may bypass restrictions
- Broken API semantics if clients silently fail without proper error handling

Remediation:

- Remove or mask the Server response header to prevent version disclosure
- Explicitly reject X-HTTP-Method-Override and X-Method-Override headers
- Return consistent, generic error bodies that do not reveal architecture details
- Upgrade gunicorn to the latest stable release and patch regularly

Risk 2 — Overly Permissive CORS Policy

Category	Severity	Affected Component	CVSS Score	Evidence
Access Control	HIGH	HTTP Response Headers	6.5 (Medium-High)	Images 3, 4

Description:

The API returns Access-Control-Allow-Origin: * combined with Access-Control-Allow-Credentials: true. This combination is a well-documented security misconfiguration. The CORS specification prohibits credentialed requests when the origin is a wildcard, but misconfigured clients or intermediary proxies may still forward cookies or tokens, enabling cross-origin data exfiltration.

Impact:

- Any web origin can make credentialed requests if a browser bug or proxy allows it
- Enables Cross-Origin Resource Sharing (CORS) abuse for data exfiltration
- Authenticated sessions may be hijacked via malicious third-party websites

Remediation:

- Never combine Access-Control-Allow-Origin: * with Access-Control-Allow-Credentials: true
- Maintain an explicit allowlist of trusted origins and reflect only those in the header
- Apply the principle of least privilege: disable credentials unless strictly required

Risk 3 — Sensitive Credential Handling in CI/CD Script

Category	Severity	Affected Component	CVSS Score	Evidence
Credential Security	HIGH	github_pull_request.sh	6.1 (Medium-High)	Image 1

Description:

The CI/CD shell script constructs a raw GitHub raw-content URL using \$GITHUB_REPOSITORY and \$GITHUB_PULL_REQUEST environment variables and fetches it via curl without certificate pinning or integrity verification. The script also obfuscates a URL scheme by splitting the string (DUMMY_SCHEME="https"), a technique that can evade static secret scanners but may mask actual credential leakage in logs.

Impact:

- URL construction via environment variables is susceptible to injection if variable values are attacker-controlled
- curl without --fail or checksum verification allows silent download of tampered content
- Obfuscated scheme string may evade secret-scanning tools, creating a false sense of security
- If GITHUB_TOKEN is inadvertently available, it could be exfiltrated via the constructed URL

Remediation:

- Validate and sanitize all environment variables before interpolating into shell commands
- Use curl --fail-with-body and verify downloaded file integrity (e.g., SHA256 checksum)
- Add secret-scanning coverage for obfuscated patterns; do not rely solely on literal matching
- Restrict GitHub Actions workflow permissions to the minimum required (read-only)

Risk 4 — Server Version Disclosure

Category	Severity	Affected Component	CVSS Score	Evidence
Information Disclosure	MEDIUM	HTTP Server Header	5.3 (Medium)	Images 3, 4

Description:

Every HTTP response includes the header Server: gunicorn/19.9.0, disclosing the exact web server software and its version. This information assists attackers in identifying applicable CVEs and tailoring exploits for this specific gunicorn release without requiring any brute-force enumeration.

Impact:

- Enables targeted reconnaissance: attackers can search CVE databases for gunicorn 19.9.0
- Reduces the time and effort required for exploit development
- Combined with other findings, builds a detailed attack surface profile

Remediation:

- Configure gunicorn or a reverse proxy (nginx/Caddy) to suppress or replace the Server header
- Upgrade to gunicorn 23.x (current stable) to benefit from security patches
- Apply WAF rules to strip version tokens from outbound headers

Risk 5 — Insufficient Input Validation in Link Validator

Category	Severity	Affected Component	CVSS Score	Evidence
----------	----------	--------------------	------------	----------

Input Validation	MEDIUM	scripts/validate/links.py	4.3 (Medium)	Image 1
------------------	--------	---------------------------	--------------	---------

Description:

The CI script pipes diff output through egrep to filter additions and passes the resulting file directly to the Python link validator. No explicit sanitization of URLs is performed before validation, and the validator is invoked by the CI runner with no sandboxing. A malicious PR contributor could craft a URL that triggers SSRF, open-redirect, or DNS rebinding against the CI runner's internal network.

Impact:

- Server-Side Request Forgery (SSRF) via crafted URLs in PR additions
- Potential access to internal CI infrastructure metadata endpoints (e.g., AWS IMDS)
- Denial-of-service against external hosts by submitting high volumes of valid-looking links

Remediation:

- Validate URL schema and hostname against an allowlist before fetching
- Block private IP ranges (RFC 1918, loopback, link-local) in the validator
- Run the validator in a sandboxed container with no access to internal networks
- Limit per-PR link count and apply rate limiting to avoid abuse

4. Consolidated Remediation Roadmap

#	Remediation Action	Severity	Effort	Timeline
1	Remove/mask Server header; upgrade gunicorn	CRITICAL	Low	Immediate (< 1 week)
2	Fix CORS: replace wildcard origin with allowlist	HIGH	Low	Immediate (< 1 week)
3	Harden CI script: validate env vars, add curl integrity checks	HIGH	Medium	Short-term (2–4 weeks)
4	Block method override headers; standardise error responses	MEDIUM	Low	Short-term (2–4 weeks)
5	Sandbox link validator; implement SSRF mitigations	MEDIUM	High	Medium-term (1–2 months)

5. Conclusion & Recommendations

The public-apis infrastructure exhibits several security weaknesses that collectively elevate the overall risk posture to HIGH. The most urgent issues — CORS misconfiguration and server version disclosure — require minimal engineering effort and should be addressed within one week. The CI/CD script hardening and SSRF mitigations require more planning but are equally important for protecting the integrity of the build pipeline.

It is strongly recommended that the repository maintainers implement automated security scanning (e.g., Dependabot, CodeQL, Semgrep) as part of the CI pipeline to catch future regressions. A follow-up penetration test should be scheduled within 90 days of remediation to verify effectiveness.

Key Recommendations

- Treat CORS wildcard + credentials as a P0 fix — patch within 24 hours
- Upgrade gunicorn and suppress the Server header at the reverse-proxy layer
- Integrate SAST/DAST tools into the GitHub Actions workflow
- Schedule a follow-up assessment within 90 days post-remediation

Prepared by: Security Analysis Team | Date: May 2026 | Version: 1.0